HelixCode — Open Issues Tracker

> Per Constitution §11.4.15 (Item-status tracking) + §11.4.16 (Item-type tracking) + §11.4.19 (Fixed-document column-alignment) + CONST-057 (Type-aware closure vocabulary) + CONST-058 (Reopened-source attribution).
>
> **Authoritative resumption ledger**: `docs/CONTINUATION.md` (CONST-044). This file complements it with item-level granularity for currently-open work.
>
> **Status vocabulary** (closed set): `Queued` | `In progress` | `Ready for testing` | `In testing` | `Reopened` | `Fixed/Implemented/Completed (→ Fixed.md)`
>
> **Type vocabulary** (closed set): `Bug` | `Feature` | `Task`

Prefix convention

Round 189 (2026-05-19) introduced per-project / per-submodule ID prefixes replacing the legacy `ISSUE-NNN` flat namespace. New items MUST use the prefix matching their scope; cross-cutting items affecting two or more submodules (or any meta-repo concern) live under `HXC`. Numeric portion is per-prefix (each prefix starts at `001`). Legacy `ISSUE-NNN` IDs renamed forward-only per CONST-043; historical close-out narratives in `docs/CONTINUATION.md` preserve original IDs verbatim, and `docs/Fixed.md` annotates each closure with `(ex-ISSUE-NNN)` for git-history traceability.

Prefix	Scope	Source
HXC	HelixCode root project (project-wide, multi-submodule, governance, infrastructure)	this repo
HXA	HelixAgent submodule	`dependencies/HelixDevelopment/HelixAgent` (when

Prefix	Scope	Source
		present) / `helix_agent/` tree
HXM	HelixMemory submodule	`dependencies/HelixDevelopment/HelixMemory`
HXL	HelixLLM submodule	`dependencies/HelixDevelopment/HelixLLM`
HXQ	HelixQA submodule	`dependencies/HelixDevelopment/HelixQA` (`helix_qa/`)
HXS	HelixSpecifier submodule	`dependencies/HelixDevelopment/HelixSpecifier`
HXO	HelixOrchestrator (= LLMOrchestrator) submodule	`dependencies/HelixDevelopment/LLMOrchestrator`
HXV	HelixVerifier (= LLMsVerifier) submodule	`dependencies/HelixDevelopment/LLMsVerifier`
HXD	HelixDocProcessor (= DocProcessor) submodule	`dependencies/HelixDevelopment/DocProcessor`
HXI	HelixI18n (when added)	tba
PLN	Planning submodule (vasic-digital)	`dependencies/vasic-digital/Planning`
VEN	VisionEngine submodule (HelixDevelopment)	`dependencies/HelixDevelopment/VisionEngine`
SLF	SelfImprove submodule (vasic-digital)	`dependencies/vasic-digital/SelfImprove`
STO	Storage submodule (vasic-digital)	`dependencies/vasic-digital/Storage`
OPS	LLMOps submodule (vasic-digital)	`dependencies/vasic-digital/LLMOps`
VDB	VectorDB submodule (vasic-digital)	`dependencies/vasic-digital/VectorDB`
OBS		

Prefix	Scope	Source
	Observability submodule (vasic-digital)	`dependencies/vasic-digital/Observability`
MCP	MCP_Module submodule (vasic-digital)	`dependencies/vasic-digital/MCP_Module`
MSG	Messaging submodule (vasic-digital)	`dependencies/vasic-digital/Messaging`
MDW	Middleware submodule (vasic-digital)	`dependencies/vasic-digital/Middleware`
PLG	Plugins submodule (vasic-digital)	`dependencies/vasic-digital/Plugins`
STR	Streaming submodule (vasic-digital)	`dependencies/vasic-digital/Streaming`
WAT	Watcher submodule (vasic-digital)	`dependencies/vasic-digital/Watcher`
CNV	conversation submodule (vasic-digital)	`dependencies/vasic-digital/conversation`
AUT	Auth submodule (vasic-digital)	`dependencies/vasic-digital/Auth`
LZY	Lazy submodule (vasic-digital)	`dependencies/vasic-digital/Lazy`
ATP	AutoTemp submodule (vasic-digital)	`dependencies/vasic-digital/AutoTemp`
PLI	PliniusCommon submodule (vasic-digital)	`dependencies/vasic-digital/PliniusCommon`
CHL	challenges submodule (vasic-digital)	`challenges/`
CNT	containers submodule	`containers/`
SEC	security submodule	`security/`
PAN	panoptic submodule	`panoptic/`

For submodules not listed above, default to the first 3 letters of the submodule name, uppercase (e.g. `Watcher` → `WAT`). Document the new prefix in this table on first use.

Legacy → new mapping (round 189)

Old ID	New ID	Scope rationale
ISSUE-001	VEN-001	VisionEngine `helix-gitlab` URL
ISSUE-002	VEN-002	VisionEngine `vasic-digital-github` fork divergent
ISSUE-003	HXL-001	HelixLLM `analysis_test.go` hardcoded path
ISSUE-004	HXL-002	HelixLLM TOON `WriteTOON` 500
ISSUE-005	HXC-001	CONST-052 rename programme (project-wide)
ISSUE-006	HXC-002	Round-74 residual LOGIC FAILs (multi-submodule cross-cutting)
ISSUE-007	HXC-003	CONST-046 migration backlog (project-wide)
ISSUE-008	HXQ-001	helix_qa `TestPerformance` flake
ISSUE-009	HXA-001	helix_agent handler tests (4)
ISSUE-010	HXA-002	helix_agent debate API drift
ISSUE-011	HXA-003	venice CONST-037 (in helix_agent)

VEN-001 (ex-ISSUE-001) — VisionEngine `helix-gitlab` remote repo missing (404) — CLOSED (→ Fixed.md)

Status: Completed (→ Fixed.md) **Type:** Task **Discovered:** 2026-05-19 (round 98 — Planning + VisionEngine i18n migration) **Discovered-By:** AI subagent during 4-remote push attempt **Closed-By:** Round 188 (subagent repo-inventory sweep) **Root cause:** The `helix-gitlab` remote URL in `dependencies/HelixDevelopment/VisionEngine/.git/config` pointed at `git@gitlab.com:HelixDevelopment/visionengine.git` — a non-existent group path. The actual GitLab group is `helixdevelopment1` (path) / `HelixDevelopment` (display name). The repository `helixdevelopment1/VisionEngine` (id 80411994) already existed since 2026-03-19. NOT a missing-repo issue — a URL-misconfiguration issue. **Fix:** `git remote set-url helix-gitlab git@gitlab.com:helixdevelopment1/VisionEngine.git` in the VisionEngine submodule, then `git push helix-gitlab master` (FF-safe: local was 46 commits ahead, remote 0 ahead). Push landed at SHA `2d0c35b` (verified via `git ls-remote helix-gitlab master`). The Upstreams recipe `push-helix-gitlab.sh` references the remote by name (not URL), so it continues to work unchanged. **Evidence:** - `glab api projects/helixdevelopment1%2Fvisionengine` → id 80411994 OK -

`git ls-remote helix-gitlab HEAD` → `2d0c35bebb199a9a199fbf899eaeb292e38eaf17` (matches local HEAD) - Original broken URL still 404s when probed directly (proves URL was the issue, not perms)

VEN-002 (ex-ISSUE-002) — VisionEngine `vasic-digital-github` fork lineage divergent at SHA 93c830a

Status: Queued — BLOCKED on operator (CONST-061 merge-first investigation) **Type:** Bug **Discovered:** 2026-05-19 (round 98) **Discovered-By:** AI subagent during 4-remote push attempt **Evidence:** vasic-digital-github HEAD `93c830a` carries round-48/52/57 commits absent from HelixDevelopment local main. Non-FF push rejected. NO force-push attempted (CONST-043). **Resolution path:** Operator-led CONST-061 merge-first pipeline — fetch divergent commits, audit conflict surface, integrate or document divergence as intentional fork, then either FF-push or designate one lineage as canonical.

HXL-001 (ex-ISSUE-003) — HelixLLM `internal/agents/tools/analysis_test.go` hardcoded absolute path

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 95 — HelixLLM migration; surfaced as pre-existing failure) **Discovered-By:** AI subagent during HelixLLM standalone test run **Closed-By:** Round 105 (commit `a5e56d4` in HelixLLM; meta pointer `fedd152`) **Attribution correction:** Originally documented as helix_agent; actual location is HelixLLM submodule (`dependencies/HelixDevelopment/HelixLLM/internal/agents/tools/`). Commit SHAs `0a84310` resolved there. **Resolution:** Replaced hardcoded path with `t.TempDir()` + 2 synthesised fixture files. Bonus: same bug-pattern discovered in `git_test.go` (constant `helixLLMRoot` + 7 tests) — refactored `gitSandbox()` signature. 6 tests now PASS on any host. Mutation verified.

HXL-002 (ex-ISSUE-004) — HelixLLM `internal/gateway/middleware` TOON `WriteTOON` returns 500

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 95) **Discovered-By:** AI subagent **Closed-By:** Round 105 (commit `a5e56d4`) **Attribution correction:** Originally documented as helix_agent; actual location is HelixLLM submodule. Commit `6f11c56` resolved there. **Resolution:** Root cause was vasic-digital/TOON's round-27 anti-bluff change (Marshal returns `ErrTOONEncodingNotImplemented` unconditionally) combined with `WriteTOON` treating ANY Marshal error as 500. Fix: fall back to `json.Marshal` while preserving `application/toon` Content-Type (matches ContentNegotiation middleware). 500 still returned for genuinely unmarshallable values (channels). 19 middleware tests now PASS. Mutation verified.

HXC-001 (ex-ISSUE-005) — CONST-052 rename programme: meta-repo directories still PascalCase

Status: Queued **Type:** Task **Discovered:** 2026-05-15 (CONST-052 cascade landed) **Discovered-By:** Constitution **Evidence:** Meta-repo directories like `helix_code/`, `challenges/`, `helix_qa/`, `helix_agent/` still PascalCase despite CONST-052 mandating snake_case. Renames deferred because they break path-encoded references throughout governance docs, CI scripts, and tracker URLs. **Resolution path:** Phased migration per CONST-052 §11.4.29: comprehensive brainstorming → phase-divided plan → fine-grained tasks → every change covered by every applicable test type. Round 88 made partial progress (3 submodules with drift fixed) but root directories remain.

HXC-002 (ex-ISSUE-006) — Round-74 residual LOGIC-class FAILs (CLOSED)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 74 — release-gate-test.sh creation; classified by round 89) **Discovered-By:** AI release-gate sweep **Closure progress:** - ✓ HelixMemory: closed round 106 (commit `69016df` — single-line `go.mod` fix; 6 FAIL → 0 FAIL) - ✓ vasic-digital/Planning: round 107 NO-OP — 275 PASS / 0 FAIL / 20 SKIP-OK; likely incidentally fixed by round 98 i18n migration - ✓ helix_agent inner: closed round 109 (commit `0f492e98` — 5 test-side bluff fixes, zero production

changes) **Evidence:** Round 74 surfaced 26 FAILs across submodules; rounds 82-87 closed 19; this Issue tracked the residual 7 across 3 submodules. All 3 components closed by rounds 106 + 107 + 109. **Follow-ups surfaced (NEW issues filed)**: 4 helix_agent handler tests previously masked by mid-run panic (now visible) + 3 build-failed packages depending on sibling submodule API drift (`digital.vasic.debate`) + 2 LOGIC FAILs reclassified as cross-cutting work (venice CONST-037 model-wiring + compliance CONST-051 architectural reconciliation). See HXA-001 through HXA-003 (filed below).

HXA-001 (ex-ISSUE-009) — helix_agent handler tests surfaced after round-109 fix

Status: Queued **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent (helix_agent LOGIC audit) **Evidence:** Mid-run panic in `TestIsProviderAvailable_NotAvailable` aborted test binary; round 109's fix unblocked execution, surfacing 4 pre-existing FAILs: `TestFormattersHandler_FormatCode_UnsupportedLanguage`, `TestEmbeddingHandler_WithRealManager`, `TestGetTaskResources`, `TestGetTaskLogs`. Out of round-109's 5-fix cap. **Resolution path:** Per-handler investigation, similar to round 109's test-side bluff pattern.

HXA-002 (ex-ISSUE-010) — helix_agent 3 build-failed packages (sibling submodule API drift)

Status: Queued — BLOCKED on cross-submodule coordination **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent **Evidence:** 3 packages in helix_agent depend on `digital.vasic.debate` API surface that changed; build fails with type/method mismatches. Pre-existing. **Resolution path:** Either rebuild the consuming code to new debate API OR pin older debate version in helix_agent go.mod. Cross-submodule coordination required.

HXA-003 (ex-ISSUE-011) — venice `TestGetCapabilities` model-list drift (CONST-037)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 109) **Discovered-By:** AI subagent **Closed:** 2026-05-19 (round 190) **Closure-Ref:** helix_agent commit (round-190 venice CONST-037 model-list drift) + meta-repo pointer-bump **Evidence:** Test hardcoded `venice-uncensored`; Venice API returned 75 models with the family rotated to `venice-uncensored-1-2` / `venice-uncensored-role-play`. Per CONST-037 (LLMsVerifier is the single source of truth for model metadata) the assertion violated the no-hardcoded-list rule. **Resolution:** `helix_agent/internal/llm/providers/venice/venice_test.go::TestGetCapabilities` — replaced `assert.Contains(..., "venice-uncensored")` and `assert.Contains(..., "llama-3.3-70b")` with structural assertion: `NotEmpty(SupportedModels)` plus a substring scan for the `venice-uncensored*` family. SKIP-OK marker per CONST-035 fires if the entire family disappears (avoids false-positive PASS). Mutation-verified (revert → FAIL with the original drift, restore → PASS).

HXC-004 — Recovery-batch under-verification (40% FAIL rate per round-193 audit)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 193 — recovery-batch verification audit) **Discovered-By:** AI subagent **Fixed:** 2026-05-19 (round 200 — per-package test-assertion repair) **Evidence:** Round-193 audit of 10 recovery-batch-landed packages (recovery commits `b7f8672` + `5c94696`) found 6 PASS / **4 FAIL**: - `internal/llm` (round 161): test-assertion drift — tests still expected pre-i18n English literal “api_key”, production emits message-ID `internal_llm_wizard_anthropic_apikey_required` - `internal/logo` (round 163): same drift — tests expected “failed to open” / “failed to decode”, production emits `internal_logo_open_source_failed` / `internal_logo_decode_source_failed` - `internal/notification` (round 167): same drift — tests expected Title literals “Task Completed”, “Task Failed”, “Workflow Completed”, “Workflow Failed”, “Worker Disconnected”, “System Error”, “System Started”; production emits `internal_notification_title_*` IDs - `internal/performance` (round 168): build break — `translator.go` `stdctx.Context` vs plain `context` — fixed inline by parent agent **Root cause:** Recovery-batch commits captured stalled-agent file content but did NOT re-run consuming-test updates + did NOT verify build/test green per-package. **Resolution:** Round-200 subagent updated test assertions in all 3 drifted packages to expect message-ID echoes

(`internal_<pkg>_*` prefix). Per-package PASS confirmed (llm: 51.8s, logo: 0.07s, notification: 0.89s, performance: 8.4s). Per CONST-035 mutation-verified one assertion per package: revert to literal → FAIL (production emits the ID), restore → PASS. **Audit reference:** `docs/audits/2026-05-19-recovery-batch-verification.md` (commit `1badef1`).

HXC-003 (ex-ISSUE-007) — CONST-046 migration backlog (57,329 violations baselined; shrinking)

Status: In progress **Type:** Feature **Discovered:** 2026-05-19 (round 92 — audit script) **Discovered-By:** AI subagent ground-truth scan **Evidence:** Round-92 scan reported 57,345 violations across 21,937 files. Round 99b baseline collapsed to 54,803 unique `(path, literal_hash)` keys. Phase 4 (rounds 100+) systematically migrating top-concentration files: round 100 (evaluators.go), 101 (challenge_recorded_ai_testgen.go), 102 (challenge_desktop.go) — see CONTINUATION.md close-outs. **Resolution path:** Continued Phase 4 cadence; audit-gate `--fail-on-new` already enforced; each migration round MUST re-run `--update-baseline` so snapshot shrinks toward zero.

HXQ-001 (ex-ISSUE-008) — helix_qa intermittent TestPerformance flake (host-load-sensitive)

Status: Queued — BLOCKED on operator (host topology decision) **Type:** Bug **Discovered:** 2026-05-19 (round 82) **Discovered-By:** AI subagent **Evidence:** helix_qa TestPerformance fails intermittently under high host load (concurrent containers + builds). Not a code bug per se; a sensitivity issue. **Resolution path:** Either (a) loosen timing tolerance with explicit comment + reference to host topology, or (b) gate the test behind a `HOST_LOAD_DEDICATED=1` env var to run only on quiescent hosts. Operator decision needed.

HXC-005 — `cmd/performance_optimization_standalone/main.go` is a CONST-035 simulation bluff

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-20 (round 317 — cmd i18n migration subagent) **Discovered-By:** AI subagent — refused to localize the file because doing so would polish a bluff **Fixed:** 2026-05-20 (round 318) **Evidence:** `helix_code/cmd/performance_optimization_standalone/main.go` was a `package main` that printed "🚀 Starting HelixCode Production Performance Optimization" then *simulated* every optimization phase: `// Simulate production optimization phases`, `time.Sleep(500 * time.Millisecond)` per phase, and `improvement := 5.0 + rand.Float64()*20.0` — fabricated improvement percentages from a random number generator. No real profiling, no real optimization, no real measurement. The canonical BLUFF-001-class anti-pattern (CLAUDE.md §3.3 / §6 ANTI-PATTERN 1) — a binary that reports success for work it never performed. Violated CONST-035 / Article XI §11.9. **Resolution:** Decision — **DELETE** (resolution path b). The standalone tool was genuinely obsolete: fully superseded by `cmd/performance_optimization/` (snake_case post-CONST-052), which calls the REAL `dev.helix.code/internal/performance.PerformanceOptimizer` — real `runtime.ReadMemStats`, real `GOMAXPROCS` tuning, real before/after measurement — and carries CONST-046 i18n + a unit-test file. `git rm -r cmd/performance_optimization_standalone/` removed the dead bluff; stale references purged from `docs/COMPREHENSIVE_AUDIT_REPORT.md`. Reproduce-before-fix Challenge added at `cmd/performance_optimization/bluff_regression_test.go`: `TestHXC005_BluffStandaloneDirectoryDeleted` asserts the obsolete path is gone + the real command survives; `TestHXC005_RealOptimizerMeasuresActualMemory` allocates a retained 32 MiB buffer and asserts the optimizer's baseline `MemoryUsage` tracks a genuine `runtime.HeapAlloc` reading (not an RNG single-digit). Post-fix evidence: `go build ./cmd/...` exit 0; `go test -count=1 -run TestHXC005 ./cmd/performance_optimization/` → both PASS (literal log: `optimizer baseline MemoryUsage=33812624 bytes, runtime.HeapAlloc=33802008 bytes — both real measurements, same order of magnitude. No RNG-fabricated improvement percentages.`); anti-bluff smoke on the deleted path returns N/A (directory gone).

PAN-001 — panoptic `appendJSONString` truncates multi-byte UTF-8 runes to bytes (`TestResult.MarshalJSON` corrupts non-ASCII)

Status: Fixed (→ Fixed.md) **Type:** Bug **Discovered:** 2026-05-19 (round 298 — panoptic enrichment subagent) **Discovered-By:** AI subagent runner-detector against real `executor.TestResult.MarshalJSON` **Fixed:** 2026-05-19 (round 302 — panoptic submodule commit `24aa627` + meta pointer bump) **Evidence:** `panoptic/internal/executor/executor.go:120` — `buf = append(buf, byte(r))` in the `else` branch of `appendJSONString` casts a `rune` to a single byte. Multi-byte UTF-8 codepoints (German umlauts, Spanish accents, Japanese CJK, Serbian Cyrillic, Chinese Han) get truncated to one byte each, producing corrupted JSON output. Honestly tracked via the round-298 Challenge runner's `executor-marshal:utf8-detector:regression-present` PASS line + KNOWN-ISSUE entry in `panoptic/docs/test-coverage.md` §7. Affects every consumer that JSON-marshals a `TestResult` containing non-ASCII text. **Resolution:** Replaced `buf = append(buf, byte(r))` with `buf = utf8.AppendRune(buf, r)` (Go 1.21+) + added `unicode/utf8` import. Single-line functional fix. Post-fix evidence: `go test -race -count=1 ./internal/executor/...` → ok 4.470s; `bash challenges/panoptic_describe_challenge.sh` → 39/39 PASS, 0 FAIL; runner UTF-8 detector flipped from `regression-present` → `fixed` (literal log: `PASS [executor-marshal:utf8-detector:fixed]` + `KNOWN-ISSUE-RESOLVED: executor.appendJSONString now UTF-8 clean`). Closed in this round.

Last regenerated: 2026-05-19 (round 302 — PAN-001 closed). To update Issues_Summary.md mechanically, run `scripts/generate_issues_summary.sh` (TODO: create — currently this Issues.md is the source of truth and Summary is hand-maintained).